

Part 5

Tech-Debt Audit and Remediation Plan

Every verified defect with reproduction, severity and evidence; a scored and categorised debt register; and a phased plan that maps onto the Plane tasks and the team's working agreements.

AUDIENCE

Whole team, especially whoever plans cycles and reviews PRs. Also anyone writing about results.

COVERS

audit method, headline findings, verified defect catalogue F-01..F-34, fixture x target validity matrix, debt register by category, priority scoring, phased remediation, mapping to T1-T11, proposed new tasks, working agreements

REPOSITORY STATE

branch main lineage, commit d1ab454

GENERATED

2026-09-20

SOURCE

docs/handbook/source/05-tech-debt-audit.md

Contents

Audit Method and Headline Findings	4
How this audit was done	4
Headline findings	4
Verified Defect Catalogue	6
F-01: Proof source side ignores dataflow, operand order and constants	6
F-02: Payload inference has no multiply-only or max-only branch	6
F-03: Loop step is parsed and thrown away	7
F-04: Induction-variable roles and constants matched by substring	7
F-05: Multi-function modules are merged into one loop nest	7
F-06: Z3 recursive-function names collide; large reductions never verify; CI red	7
F-07: Output initial value and max sentinel not modelled	8
F-08: Emitted Linalg syntax is invalid for map, reduce, dot and conv_1d	8
F-09: StableHLO artifacts are never actually validated	8
F-10: Trusted lane accepts a fake verifier	8
F-11: --no-verify does not skip verification	9
F-12: Result states conflate failures with refutations	9
F-13: Strict CLI exit code 2 is unreachable with the real lifter	9
F-14: Parametric proof pilot overstates what it proves	9
F-15: No independent numerical oracle	9
F-16: min/max affine bounds become opaque symbol strings	10
F-17: replay command crashes on reports without a Z3 verdict	10
F-18: Structural pre-filter has no rule for SCALE, NEGATE, MAX, MIN	10
F-19: Candidate ranking lets the convolution recogniser override confidence	10
F-20: Accumulation detection treats any add as accumulation	10
F-21: Silent information loss in the parser	11
F-22: MLIR syntax validation depends on an undeclared optional package	11
F-23: Duplicated logic across modules	11
F-24: cli.py is a 1,500-line module with inconsistent option handling	11
F-25: Dead and misleading code	12
F-26: Test data shipped inside the runtime package	12
F-27: Documentation contradicts code and itself	12
F-28: Metrics and evidence pipeline measures the tool's opinion of itself	12
F-29: Tests that cannot fail, or test the mock	13
F-30: CI covers one Python version and no static checks	13

F-31: Docker build is not reproducible as documented	13
F-32: Dependencies unpinned and declared twice	13
F-33: Repository hygiene	14
F-34: Text-based MLIR handling and real-number semantics (structural limits)	14
Fixture x Target Validity Matrix	15
Linalg target	15
StableHLO target	16
Debt Register by Category	17
Code debt	17
Architecture debt	17
Test debt	17
Dependency debt	17
Documentation debt	18
Infrastructure debt	18
Process debt	18
Prioritised Register	19
Remediation Plan	21
Phase 0: Stop the bleeding (weeks 1 to 2)	21
Phase 1: A sound verifier (weeks 2 to 6)	22
Phase 2: Valid artifacts (weeks 6 to 10)	22
Phase 3: Maintainability (ongoing, alongside features)	23
Proposed new Plane tasks (need confirmation before creation)	23
Working Agreements to Keep the Plot	24
Appendix: Reproducing the Findings	25

Audit Method and Headline Findings

How this audit was done

1. **Read everything that executes.** All 12 library modules (about 9,000 lines), all 27 test files (365 tests), `conftest.py`, the scripts, the example demo, the Dockerfile, `toolchain.lock` and the three GitHub workflows.
2. **Read the main documents** (setup, status, audit, plans, lane indexes, ADR, papers' findings file) and compared their claims with the code.
3. **Reconstructed the history** from `git log` (37 commits), commit sizes, per-file churn and the CI run history (`gh run list`, failing log of run 34763839671).
4. **Ran the test suite** in a clean virtual environment with the declared dependencies (Python 3.13, z3-solver 5.1.0, sympy 1.14.0): 364 passed, 1 failed, identical to CI.
5. **Ran targeted experiments** for each suspected defect, including mutated kernels, a verifier-discovery test with Docker disabled, a replay of the failing test's Z3 notes, and a full matrix of all 37 kernels in `ALL_FIXTURES` times both targets, validating every emitted artifact with the real `mlir-opt` 17.0.6 inside the project's Docker image.
6. **Cross-checked with the Plane board.** Tasks T1 to T11 (LOOPH-9 to LOOPH-19) were written from an earlier September review. This audit confirms their findings independently, adds new ones, and marks which findings are already tracked.

Scoring follows the tech-debt framework: **Impact** (how much it slows or misleads the team, 1 to 5), **Risk** (what happens if it is not fixed, 1 to 5), **Effort** (1 to 5, lower is easier) and **Priority = (Impact + Risk) x (6 - Effort)**.

Headline findings

1. "FORMALLY PROVED" does not mean the lifted code is equivalent

The Z3 encoding reconstructs the source program's arithmetic from a coarse category instead of from its operations. Verified: `C = A*B + A` proved as elementwise multiply; `C = B - A` proved as `A - B`; `c += 2*A[k]*B[k]` proved as plain dot; loops with `step 2` proved as full-tensor ops; the canonical `elementwise_mul` fixture proved as a scalar-scale op with an invented `%alpha` parameter; `relu` proved as a plain copy. (F-01 to F-05)

2. Most emitted artifacts are not valid MLIR, and StableHLO cannot be validated at all

Of 31 Linalg artifacts the lifter marks PROVED across the fixture set, 12 are rejected by `mlir-opt` 17: every `linalg.map` and `linalg.reduce` (syntax), `linalg.dot` and `linalg.conv_1d_ncw_fcw` (rank). All 29 PROVED StableHLO artifacts fail validation because the image's `mlir-opt` has no StableHLO dialect. (F-08, F-09)

3. The quality gates are weaker than documented

The trusted lane accepts a brace-counting fallback "verifier" that approves arbitrary text; `--no-verify` removes the Z3 time limit instead of skipping Z3; parse failures and encoding crashes are reported as REFUTED and then excused as "expected"; CI on `main` has been red since the restructure because of a Z3 name collision that disables every proof with more than 32 reduction iterations. (F-06, F-10 to F-13)

4. Documentation and metrics are not a reliable source of truth

At least eight different "current" test counts; sketch and fixture counts that are two versions old; a proof-quality grade computed from the unsound prover's own verdicts; papers citing report files that are not in git. (F-27, F-28)

5. The debt is concentrated and fixable

The soundness problems live in about 400 lines of `z3_checker.py` and 150 lines of `affine_extractor.py`. The emitter syntax problems are template strings. The gate problems are a few conditionals. The larger structural debts (duplicated logic, a 1,500-line CLI, text-based parsing) can be paid down incrementally once correctness is restored.

Verified Defect Catalogue

Each finding lists what is wrong, where, how it was reproduced, severity, and whether a Plane task already covers it. "New" means not covered by T1 to T11.

F-01: Proof source side ignores dataflow, operand order and constants

- **Severity:** Critical. **Category:** code (correctness). **Tracked:** T5 (LOOPH-13), regression cases in T1.
- **Where:** `z3_checker.py _build_rhs_expr` (777), `_build_elementwise_expr` (804-845), `_body_term_at` (1170-1199), `_recfunc_reduction` (1201-1260). The source term is `payload(first read, second read)`, where the payload is inferred from the `set` of op kinds.
- **Reproduction** (4x4 elementwise kernels, `Lifter(target=..., strict_mode=True).lift(src)`):

```
C = A*B + A      (mulf then addf %m, %a)  linalg -> PROVED linalg.map{arith.mulf}  stablehlo -> PROVED
  stablehlo.multiply
C = B - A        (subf %b, %a)           linalg -> PROVED linalg.map{arith.subf}  (emits A - B)
c += 2*A[k]*B[k] linalg -> PROVED linalg.dot      stablehlo -> PROVED
  dot_general_vecdot
```

- **Why it matters:** the verification stage is the project's central claim. With this defect it verifies only that index patterns line up.

F-02: Payload inference has no multiply-only or max-only branch

- **Severity:** Critical. **Category:** code. **Tracked:** part of T5 ("anything unrecognised falls through to COPY"); the specific canonical-fixture failure is new.
- **Status:** mitigated by LOOPH-20 (N1). `_infer_compute_payload` matches exact op-kind sets and raises on anything unmodelled; elementwise payloads need exactly the reads they model; RELU needs a max against a constant 0.0 from a clean parse. `ELEMENTWISE_MUL_MLIR` now lifts to `linalg.map{arith.mulf}` / `stablehlo.multiply` and `RELU_MLIR` to `linalg.map{arith.maxf_zero}`. A sweep of every sketch over every fixture changed no other fixture's verdict, and seven mutant fixtures (div, negate, scale by constant, add constant, three inputs, mul-add, hex constant) have no EQUIVALENT verdict from any sketch. Still proved wrongly, pending T5: `C = B - A` as `A - B`; `C = A*B + A` (one load of A, used twice) as elementwise mul; and `C += 2*A*B` as plain matmul. The payload model sees which op kinds occur, not how often or on which values.
- **Where:** `z3_checker.py _infer_compute_payload` (1644-1668). `{mulf}` alone, `{divf}`, `{negf}` fall through to `COPY`; `{maxf}` with one read returns the read unchanged in `_build_elementwise_expr`.
- **Reproduction:** the unmodified fixture `ELEMENTWISE_MUL_MLIR` is lifted as `linalg.map{arith.mulf_scalar}`. The correct sketch `linalg.map{arith.mulf}` is tried first and REFUTED (source modelled as `C == A`), then the scale sketch (also `C == A`) is "proved". The unmodified `RELU_MLIR` is lifted as `linalg.copy` (max with zero dropped). Both are marked PROVED; `tests/integration` has no assertion on which sketch these fixtures map to for Linalg.
- **Quick mitigation:** add explicit `MULTIPLY`, `MAX`, `NEGATE` branches and make unknown sets raise (so the result is `ENCODE_ERROR`). This is a one-hour change that stops the canonical-fixture false proofs even before the full T5 rewrite.

F-03: Loop `step` is parsed and thrown away

- **Severity:** Critical. **Category:** code. **Tracked:** T7 (LOOPH-15).
- **Where:** `affine_extractor.py` `_parse_affine_for` (637-638) and `_parse_scf_for` (662-663).
- **Reproduction:** the committed fixture `MIXED_REALWORLD_MATMUL_MLIR` has `scf.for %i = %c0 to %c4 step %c2`. Parsed bounds are `{'i': (0, 4), 'j': (0, 4), 'k': (0, 4)}` and it is PROVED as `linalg.matmul`. Adding `step 2` to the plain `matmul` fixture also yields PROVED. The test `test_lift_mixed_realworld_matmul_fixture` asserts this wrong result as success.

F-04: Induction-variable roles and constants matched by substring

- **Severity:** High. **Category:** code. **Tracked:** T3 (LOOPH-11).
- **Where:** `affine_extractor.py` `_classify_iv_roles` (1234-1239, `if iv in expr`); `_apply_known_constants` (265-279) also replaces the %-stripped name, so a constant `%1` rewrites every literal `1`; the same substring test is in `sympy_tracer.py` (437, 446, 474).
- **Reproduction:** a column-sum with IVs `%i` and `%i2` classifies both as parallel ("`i`" in "`i2`"), no sketch verifies, state REFUTED. Renaming `%i2` to `%jj` gives `reduction: ['i']` and PROVED `linalg.reduce{arith.addf}_colsum`. Polygeist's numeric names (`%arg1` vs `%arg10`, `%1`) make this a practical problem, not a curiosity.

F-05: Multi-function modules are merged into one loop nest

- **Severity:** High. **Category:** code. **Tracked:** T4 (LOOPH-12).
- **Where:** `affine_extractor.py` `extract` (345-390) takes the function name from the first `func.func` but collects loops and accesses from the entire text. `loophole compile` passes `--function=*`, so real C files routinely produce multi-function modules.

F-06: Z3 recursive-function names collide; large reductions never verify; CI red

- **Severity:** High. **Category:** code and infrastructure. **Tracked:** T2 (LOOPH-10).
- **Status:** fixed by LOOPH-10. `RecFunction` names get a process-wide serial suffix (`_fresh_rec_name`); sketch/tensor rank mismatches raise a named `ENCODE_ERROR` (`_check_operand_rank`); the lifter names candidates that reached no verdict instead of reporting "failed Z3 verification". Re-checking every fixture changed two verdicts (`MATMUL_128_MLIR` with `linalg.matmul` and `stablehlo.dot_general`, `ENCODE_ERROR` to `TIMEOUT`). Nothing newly proved.
- **Where:** `z3_checker.py` 1225 and 1521 build names from `func_name` and IV only; Z3 recursive definitions are process-global.
- **Reproduction:** on `MATMUL_128_MLIR` the notes are `linalg.conv_1d_ncw_fcw -> ENCODE_ERROR: Wrong number of arguments (1) passed to function (declare-fun B (Int Int) Real)` and `linalg.matmul -> ENCODE_ERROR: recursive function _acc_matmul_128_k already defined`. The lifter drops both and reports "All 2 candidates failed Z3 verification." This is exactly the failure in CI run 34763839671 on `main` (1 failed, 364 passed). Why the same test passed in April was not investigated; `z3-solver` is unpinned and a newer release is a plausible trigger.
- **Also:** the arity error above is not specific to recursive encodings. Sketch operands are bound to source tensor functions whose arity is the tensor's rank, then applied with the sketch indexing map's rank. On every path, input and output, a mismatch raised a raw Z3 error: "Wrong number of arguments" or "index out of bounds". In a sweep of every sketch over every fixture just before the fix, that accounted for 45 of the 48 `ENCODE_ERRORS`; the other 3 were the name collision.

F-07: Output initial value and max sentinel not modelled

- **Severity:** High. **Category:** code. **Tracked:** T6 (LOOPH-14).
- **Where:** source assertion encodes `C == sum` rather than `C_new == C_old + sum`; StableHLO emission returns a fresh tensor and so drops `C_old` (`emitter.py` 1164-1201); max reductions start from `RealVal(-1e30)` in seven places.
- **Consequence:** `examples/matmul_inplace.c` lifted to StableHLO is "proved" but differs unless `C` starts at zero.

F-08: Emitted Linalg syntax is invalid for map, reduce, dot and conv_1d

- **Severity:** High. **Category:** code. **Tracked:** New (T9 covers the verifier, not the emitter templates).
- **Where:** `emitter.py` `_emit_elementwise` (828-868), `_emit_relu` (870-893), `_emit_scale` (895-917), `_emit_reduce_sum` (939-965), `_emit_reduce_max` (967-990) use a non-existent region syntax (`{%a: f32, %b: f32} -> { ... }`); `_emit_dot` passes a rank-1 `memref<1xf32>` where `linalg.dot` needs a rank-0 output; `_emit_conv1d_ncw` emits rank-2 operands for an op whose indexing maps are rank 3. `arith.maxf` is used in several templates and should be checked against the target MLIR version as well.
- **Reproduction** (`mlir-opt` 17.0.6 in `loophole-polygeist:llvm17`):

```
elementwise_add -> error: expected SSA operand (line 12, the region header)
reduce_sum      -> error: expected SSA operand
dot_product     -> error: 'linalg.dot' op expected operand rank (1) to match the result rank of indexing
                 _map #2 (0)
conv1d          -> error: 'linalg.conv_1d_ncw_fcw' op expected operand rank (2) to match the result rank
                 of indexing_map #0 (3)
```

- **Why it survived:** the unit tests check strings ("`linalg`" in `text`, balanced braces). Artifact-validation tests exist only for `matmul`, `transpose` and `conv2d`, which are among the handlers that happen to be valid.

F-09: StableHLO artifacts are never actually validated

- **Severity:** High. **Category:** infrastructure. **Tracked:** T9 (LOOPH-17), second half.
- **Where:** `mlir_validator.py` always runs `mlir-opt` (Docker or PATH). LLVM 17's `mlir-opt` does not include StableHLO. Every StableHLO artifact yields `Dialect 'stablehlo' not found`. `run_batch_report_in_docker.ps1 -Target stablehlo -Profile trusted`, documented in `SETUP.md`, therefore always fails validation.

F-10: Trusted lane accepts a fake verifier

- **Severity:** High. **Category:** code (gate). **Tracked:** T9 (LOOPH-17), first half.
- **Where:** `find_mlir_verifier` (`mlir_validator.py` 35-68) never returns `None`; it ends with the internal checker (126-162), which only counts braces and looks for `func.func`, `return` and an op prefix. `cli.py` batch (1018-1032) only fails on `None`, so the "no verifier" branch is dead code in production, and the test that covers it monkeypatches the lookup to return `None`.
- **Reproduction:** with `LOOPHOLE_DISABLE_DOCKER_MLIR_VERIFY=1` and no `mlir-opt`, `find_mlir_verifier()` returns `loophole-internal-mlir-verify`, and validating `func.func @x() { linalg.this_op_does_not_exist return }` returns `ok=True`.

F-11: `--no-verify` does not skip verification

- **Severity:** Medium. **Category:** code (CLI). **Tracked:** T10 (LOOPH-18).
- **Where:** `cli.py` 420 and 750 pass `z3_timeout_ms=0`; Z3 interprets 0 as "no timeout". Reproduced: `Lifter(z3_timeout_ms=0).lift(MATMUL_MLIR)` returns EQUIVALENT after a full solve. The help text says "Skip Z3 verification (emit based on SymPy match only)".

F-12: Result states conflate failures with refutations

- **Severity:** Medium. **Category:** code and metrics. **Tracked:** T10 (LOOPH-18).
- **Where:** `LiftResult.result_state` (`lifter.py` 236-244) returns REFUTED whenever `verification is None`; `_verify_candidates` (620-675) reports ENCODE_ERROR and STRUCTURAL_MISMATCH candidates only when no candidate reaches a verdict (since LOOPH-10); otherwise they are still discarded.
- **Reproduction:** input `"this is not mlir at all"` gives state REFUTED with error "No sketch candidates passed structural pre-filter." and `func_name unknown`. The weekly report's "corpus refuted (expected) 5" are parse or no-match failures.

F-13: Strict CLI exit code 2 is unreachable with the real lifter

- **Severity:** Low. **Category:** code (CLI contract). **Tracked:** New.
- **Where:** `_print_lift_result` (`cli.py` 441-502) exits 2 for `strict_mode` and `partial_success`. But when strict mode is on, `Lifter.lift` itself rejects TIMEOUT/UNKNOWN and returns no emitted MLIR, so `partial_success` is false and the command exits 1 first. The exit-2 tests pass only because they replace `Lifter.lift` with a fake. This was established by reading the code; an end-to-end reproduction was not possible because the fixture proofs complete even with `--z3-timeout 1`. Decide on one contract and test it without mocks.

F-14: Parametric proof pilot overstates what it proves

- **Severity:** Medium. **Category:** code and documentation. **Tracked:** New.
- **Where:** `z3_checker.py _collect_shape_dims` (1676-1693) maps equal extents to the same symbol, so a 4x4x4 matmul is parametrised by a single `M` (square matrices only). Tensor shapes stay concrete, and the guarded unroll takes its upper limit from those shapes, so the proof covers sizes up to 4, not "all valid shapes". Reproduced notes: `[parametric over dims=['M']]` for both matmul and matvec.
- **Docs impact:** lane A record 07 and plan v2 section 12.14 state "all three pilot kernels achieved parametric EQUIVALENT"; this should be restated as "square, bounded by fixture size".

F-15: No independent numerical oracle

- **Severity:** High. **Category:** test. **Tracked:** T11 (LOOPH-19).
- Every correctness claim rests on the same encoder that has F-01 to F-07. Running the source loop and a NumPy implementation of the chosen op on random inputs would have caught all of them.

F-16: `min/max` affine bounds become opaque symbol strings

- **Severity:** Medium. **Category:** code (parser). **Tracked:** New.
- **Where:** `_resolve_bound` (`affine_extractor.py` 1279-1301) returns the raw text. Reproduced: `affine.for %i = 0 to min #map(%N)[%N]` yields bound `'min #map(%N)[%N]'`, which the verifier then turns into a sanitised free integer symbol. Polygeist emits such bounds for loops over `n` with guards.

F-17: `replay` command crashes on reports without a Z3 verdict

- **Severity:** Low. **Category:** code (CLI). **Tracked:** New.
- **Where:** `cli.py` 1495 `CheckResult[entry.get("z3", "UNKNOWN")]`; batch reports write `"z3": "-"` when there was no verification. Reproduced: `loophole replay report.json` exits 1 with `KeyError('-')`. Also `replay_checker.py` 242 evaluates `loop.output_tensor in k`, which raises `TypeError` when there is no output tensor.

F-18: Structural pre-filter has no rule for `SCALE`, `NEGATE`, `MAX`, `MIN`

- **Severity:** High. **Category:** code. **Tracked:** New (closely related to T5 and T8).
- **Status:** fixed by LOOPH-20 (N1). `SCALE` requires a multiply with exactly one loaded operand; `NEGATE`, `MAX` and `MIN` require their op. No library sketch uses `NEGATE`, `MAX` or `MIN` yet. The scale sketch's own semantics still ignore `%alpha` (T8).
- **Where:** `structural_match` (`z3_checker.py` 293-342) has no branch for these payloads, so any loop with the right loop counts passes. That is why the scale sketch becomes a candidate for `elementwise_mul`, `relu` and `A + 1.0`. Combined with F-02 it turns into false proofs.

F-19: Candidate ranking lets the convolution recogniser override confidence

- **Severity:** Medium. **Category:** code. **Tracked:** New.
- **Where:** `lifter.py` 565 sorts by `(conv_match, confidence)`, so a recogniser match always ranks first regardless of confidence; the comment says "when confidence ties". `recognise_convolution` returns only Linalg names, so it never helps the StableHLO target. `_match_candidates` returns `top_k * 2` candidates but only `top_k` are verified (575 vs 617).

F-20: Accumulation detection treats any add as accumulation

- **Severity:** Medium. **Category:** code. **Tracked:** New.
- **Where:** `_detect_accumulation` (`affine_extractor.py` 1265-1275): if the output tensor is also read, the presence of *any* `addf/addi` marks the kernel as accumulating, regardless of dataflow (only a debug-level diagnostic is emitted).

F-21: Silent information loss in the parser

- **Severity:** Medium. **Category:** code. **Tracked:** New.
- N-ary arithmetic truncated to two operands (1084-1093), unparseable constants replaced by 0.0 (1054-1064), element type defaulting to `f32` (1207-1217), `iter_args` accepted for `affine.for` but not modelled for `scf.for`, conditionals only warned about. Each emits a diagnostic, but the lift continues and can still be marked PROVED.

F-22: MLIR syntax validation depends on an undeclared optional package

- **Severity:** Low. **Category:** dependency. **Tracked:** New.
- **Where:** `affine_extractor.py` 16-19 and 451-469 validate input with `mlir.ir` if importable. The MLIR Python bindings are not a dependency and are not in the Docker image, so this path never runs. The module docstring says parsing is "intentionally implemented without regex-based extraction", but the module imports and uses `re`.

F-23: Duplicated logic across modules

- **Severity:** Medium. **Category:** code (maintainability). **Tracked:** New.
- `CANONICAL_STABLEHLO_SKETCHES`, `_canonicalize_stablehlo_sketch_name`, `_sha256_text`, `_compute_fixtures_fingerprint` exist in both `cli.py` and `scripts/generate_weekly_benchmark_report.py`.
- Payload inference exists twice with *different rules*: `SympyTracer._infer_cp` (mul+add without reduction gives ADD) vs `Z3EquivalenceChecker._infer_compute_payload` (gives MULTIPLY).
- The `can_unroll` computation appears three times and the reduction-range generator four times in `z3_checker.py`; the reduction encoders are repeated for source and sketch sides.
- Transpose permutation inference and metadata validation are written twice in `emitter.py`.
- Docker availability and image resolution are written three times (`polygeist_frontend.py`, `mlir_validator.py`, and `cli.py` compile with a *different default image*).
- `PolygeistFrontend.build_command` and `_build_docker_command` duplicate cgeist flag assembly.

F-24: `cli.py` is a 1,500-line module with inconsistent option handling

- **Severity:** Medium. **Category:** architecture. **Tracked:** New.
- Options such as `--target`, `--profile`, `--strict`, `--z3-timeout`, `--top-k` are redeclared per command with different defaults: `lift` and `lift-c` default `--z3-timeout` to 10000 (overriding the profile's 15000 in `ci-strict`), while `batch` and `demo` default to `None` (use the profile). `--parametric` and `--show-replay` exist only on `lift`. `compile` has no `--function` and no timeout and normalises dims with `sed`, while `lift-c` uses a different code path without that normalisation. `verify` ignores profiles and dialect.

F-25: Dead and misleading code

- **Severity:** Low. **Category:** code. **Tracked:** New.
- `_emit_generic` and `_build_generic_body` are unreachable (both emitters raise for unmapped sketches first).
- Conv and pool emitters compute `N`, `W_in`, `IH`, `IW` and similar values that are never used.
- `Lifter.lift_many`, `IteratorType.WINDOW`, `CounterexampleReport.max_abs_diff` (always `None`), many unused imports in `z3_checker.py` and `sympy_tracer.py`.
- `_verify_symbolic` simply calls `_verify_concrete`; the module docstring describes a quantified inductive path that does not exist as such.
- The version `0.1.0` is hard-coded in four places (`pyproject.toml`, `__init__.py`, `cli.py` `version_option`, emitter header comment).

F-26: Test data shipped inside the runtime package

- **Severity:** Low. **Category:** architecture. **Tracked:** New.
- `src/loophole/tests/fixtures.py` is part of the installed package, and `loophole demo` and `examples/run_demo.py` import it. There are two different `tests` packages (`src/loophole/tests` and `packages/loophole/tests`). Move demo kernels to `loophole/demo_kernels.py` (or package data) and keep tests out of `src`.

F-27: Documentation contradicts code and itself

- **Severity:** High. **Category:** documentation. **Tracked:** New (partly addressed by this handbook).
- See Part 1 chapter 6. Key items: eight different test counts; 26 vs 35 sketches; 10 vs 39 fixture kernels; "toolchain.lock pins the build" (it does not); lane READMEs pointing at `POC_initial_demo/`; the IEEE findings citing a report that is not in git; "proof quality grade B" and "unexpected refuted 0" derived from F-01/F-12.

F-28: Metrics and evidence pipeline measures the tool's opinion of itself

- **Severity:** High. **Category:** process and test. **Tracked:** New.
- The weekly and proof-quality scripts construct `Lifter` directly (no profile, no validation, `rglob` including corpus), count parse failures as REFUTED, classify `corpus/*` refutations as "expected", and grade strict acceptance. Reports are written to the git-ignored `reports/` folder and copied by hand into `research/benchmarks/baselines/`, whose JSON still contains absolute Windows paths from other machines. Until F-01, F-08 and F-12 are fixed, the grade should not be published.

F-29: Tests that cannot fail, or test the mock

- **Severity:** Medium. **Category:** test. **Tracked:** New (T1 adds the missing negative cases).
- Conditional assertions (`if result.success: assert ...`) in `test_matmul.py`, `test_conv1d.py` and others pass when the lift fails.
- CLI contract tests replace `Lifter.lift` and `find_mlir_verifier`, so they test the CLI's branching against invented results (see F-10, F-13).
- Tests reach into private methods (`_verify_parametric`, `_collect_shape_dims`, `lifter._emit`), freezing internals.
- `test_fixture_source_policy.py` greps other test files for `func.func @`.
- A15 tests accept any of five results including `ENCODE_ERROR` as "valid".
- Test classes and CI steps are named after plan task IDs (`TestEmitterReliabilityBurnDownB16`), which carries no meaning for a newcomer.

F-30: CI covers one Python version and no static checks

- **Severity:** Medium. **Category:** infrastructure. **Tracked:** New.
- `ci.yml` runs Python 3.11 only (the package claims `>=3.10`; the team uses 3.13), installs `requirements.txt` rather than the `[dev]` extra, has no lint, formatter, type check or coverage report, re-runs individual test node IDs that the full run already executed, and uses actions that emit Node 20 deprecation warnings. It validates only one fixture in the trusted batch smoke step.

F-31: Docker build is not reproducible as documented

- **Severity:** Medium. **Category:** infrastructure. **Tracked:** New.
- The Dockerfile clones Polygeist branch `llvm17bump` at HEAD with `--depth 1`, not `POLYGEIST_REF` from `toolchain.lock`; LLVM comes from Ubuntu's apt packages, not `LLVM_REF`; the base image `ubuntu:24.04` is not pinned by digest. Source patches are applied with Python `str.replace` and several are silent no-ops if upstream text changes (only some raise). The image does not install LoopHole itself, so the PowerShell wrappers `pip install` inside a fresh venv on every run. The GHCR package is private, which forces a PAT or the export workflow for collaborators.

F-32: Dependencies unpinned and declared twice

- **Severity:** Medium. **Category:** dependency. **Tracked:** New.
- `pyproject.toml` and `requirements.txt` both list `z3-solver>=4.12.0`, `sympy>=1.12`, `rich>=13.0`, `click>=8.1` with no upper bounds and no lock file. Z3 behaviour (timeouts, recursive-function handling, model printing) changes between releases and directly affects results (see F-06).

F-33: Repository hygiene

- **Severity:** Low. **Category:** infrastructure. **Tracked:** New.
- `.claude/settings.local.json` (a personal AI-assistant permission file allowing `Bash(python *)` and `Bash(del *)`) is committed.
- Generated artifacts are committed next to hand-written inputs: `tests/fixtures/*_lifted.mlir` (which `batch` overwrites when run on the fixtures directory with default options), `tests/fixtures/loophole_report.json`, the stale `corpus/parser_compat_summary.md`.
- Paper PDFs are committed (acceptable by the papers README, but they diverge from sources).

F-34: Text-based MLIR handling and real-number semantics (structural limits)

- **Severity:** Medium (long-term). **Category:** architecture. **Tracked:** New (research-level).
- The parser works line by line with string splitting; it assumes one operation per line and cannot follow regions, block arguments or `affine.yield` generally. The emitter builds MLIR with f-strings. Both would be more robust using MLIR's Python bindings or a small real parser, but that is a large change.
- Proofs are over mathematical reals, not IEEE floats or fixed-width integers. Any published claim must state this.

Fixture x Target Validity Matrix

Result of lifting all 37 kernels in `ALL_FIXTURES` with `Lifter(strict_mode=False, z3_timeout_ms=10000)` and validating every emitted artifact with `mlir-opt 17.0.6` in Docker. "mlir-opt" column: OK = parsed and verified; FAIL = rejected; n/a = nothing emitted. "Semantics" marks results that are wrong even when the syntax is valid.

Linalg target

Fixture	State	Sketch	mlir-opt	Semantics
matmul_4x4, matmul_dynamic_dims, matmul_unranked_memref, matmul_zero_dim_static, mixed_affine_scf_matmul	PROVED	linalg.matmul	OK	correct
mixed_realworld_matmul	PROVED	linalg.matmul	OK	wrong (step 2 ignored, F-03)
transpose_2d, transpose_nonsquare	PROVED	linalg.transpose	OK	correct
conv2d_simple, conv2d_1x1, conv2d_5x5, conv2d_strided_dilated, conv2d_strided_dilated_reordered, conv2d_var_times_const_index	PROVED	linalg.conv_2d	OK	correct
conv2d_nhwc	PROVED	linalg.conv_2d_nhwc_hwcf	OK	correct
matvec, matvec_tall, matvec_wide	PROVED	linalg.matvec	OK	correct
conv1d, conv1d_strided_dilated	PROVED	linalg.conv_1d_ncw_fcw	FAIL (rank)	invalid artifact
dot_product, dot_product_16, dot_symbolic_arith	PROVED	linalg.dot	FAIL (rank)	invalid artifact
elementwise_add, elementwise_sub	PROVED	linalg.map{...}	FAIL (syntax)	invalid artifact
elementwise_mul	PROVED	linalg.map{arith.mulf_scalar}	FAIL (syntax)	wrong op (F-02)
reduce_sum, reduce_sum_colwise, reduce_max, mixed_scf_affine_reduction	PROVED	linalg.reduce{...}	FAIL (syntax)	invalid artifact
relu	PROVED	linalg.copy	OK	wrong op (max dropped, F-02)
matmul_128x128	REFUTED	none	n/a	encode error, F-06
matmul_dynamic_conflicting_annotations	REFUTED	linalg.matmul	n/a	refuted
transpose_3d_perm_120, transpose_3d_perm_201, index_variation_eq_a, index_variation_eq_b	REFUTED	none	n/a	no sketch exists

Summary: 31 PROVED and 6 REFUTED. Of the 31 PROVED, 12 are invalid MLIR (one of which is also the wrong operation) and 2 more are syntactically valid but semantically wrong, leaving **17 of 37 fixtures with a correct and valid Linalg artifact**.

StableHLO target

Group	State	mlir-opt
matmul variants, transpose 2-D, conv1d/conv2d variants, dot, matvec, elementwise add/sub, reduce sum/colsum/max (29 fixtures)	PROVED	FAIL: Dialect ' stablehlo ' not found (cannot be validated)
elementwise_mul, relu	REFUTED (stablehlo.multiply refuted because of F-02; relu mapped to transpose and refuted)	n/a
matmul_128x128, 3-D transposes, index variations, conflicting annotations	REFUTED	n/a

Note that **elementwise_mul** is REFUTED for StableHLO but PROVED (as the wrong op) for Linalg: the same kernel gets contradictory verdicts depending only on which sketches are in the candidate list.

Debt Register by Category

Code debt

ID	Item	Where
F-01, F-02, F-18	Unsound proof encoding and pre-filter	z3_checker.py
F-03, F-04, F-05, F-16, F-20, F-21	Parser drops or misreads information	affine_extractor.py , sympy_tracer.py
F-07	Initial value and max sentinel	z3_checker.py , emitter.py
F-08	Invalid emitter templates	emitter.py
F-11, F-12, F-13, F-17	CLI and result-state contract	cli.py , lifter.py , replay_checker.py
F-14	Parametric pilot scope	z3_checker.py
F-19	Candidate ranking	lifter.py
F-23, F-25	Duplication and dead code	all modules

Architecture debt

ID	Item
F-24	CLI god module and per-command option drift
F-26	Test data inside runtime package
F-34	Text-based MLIR parsing and emission; real-number semantics
(Part 2)	No typed configuration object: profile, flags and eight environment variables are resolved in four different places
(Part 2)	Verification and emission are not connected by a shared semantic model: the sketch says what the op means, the emitter independently decides what text to write, and nothing checks that the two agree

Test debt

ID	Item
F-15	No numerical oracle
F-29	Vacuous, mock-heavy and private-API tests; task-ID naming
(T1)	No negative (mutant) kernels
(F-08)	Artifact validation only for three handlers

Dependency debt

ID	Item
F-22	Undeclared optional MLIR Python bindings
F-32	Unpinned, duplicated dependency lists
F-31	Unpinned Polygeist, apt LLVM, base image

Documentation debt

ID	Item
F-27	Contradictory, stale and aspirational docs
F-28	Published metrics derived from unsound verdicts
(Part 1)	No architecture document matching today's code before this handbook

Infrastructure debt

ID	Item
F-06	CI red on <code>main</code>
F-09, F-10	Validation gates
F-30	CI breadth
F-31	Reproducible image
F-33	Committed local settings and generated files

Process debt

Item	Evidence
Very large mixed commits, no review	0ec008e (316k lines), 3035d48 (43 files, "patches ... and other small patches"), 93e58bf (2,455 lines, three features)
Closure by summary document	37 worklog records, each restating test counts
Plan velocity mismatch	A 12-week plan closed in about 14 days, with tasks marked Done on the strength of their own tests
Knowledge left with people who have moved on	No current team member authored the code; the April authors' reasoning is only in worklog prose

Prioritised Register

Priority = (Impact + Risk) x (6 - Effort). The formula favours quick wins; it deliberately ranks the big T5 rewrite lower than one-line fixes. Sequencing (next chapter) overrides pure priority where dependencies exist: F-01 is the most important item even though its score is moderate.

Rank	ID	Item	Impact	Risk	Effort	Priority	Tracked
1	F-02	Payload inference fall-through to COPY	4	5	1	45	T5 (part)
2	F-04	Substring IV roles, numbered constants	4	4	1	40	T3
3	F-10	Trusted lane accepts fake verifier	3	5	1	40	T9
4	F-03	Loop step discarded	4	5	2	36	T7
5	F-06	RecFunction name collision, CI red	4	3	1	35	T2
6	F-18	Pre-filter missing SCALE/NEGATE/MAX/MIN rules	3	4	1	35	New
7	F-05	Multi-function modules merged	4	4	2	32	T4
8	F-08	Invalid Linalg emitter templates	4	4	2	32	New
9	F-09	StableHLO never validated	4	4	2	32	T9
10	F-27	Contradictory documentation	4	4	2	32	New (this handbook)
11	F-11	--no-verify removes the timeout	3	3	1	30	T10
12	F-15	No numerical oracle	4	5	3	27	T11
13	F-32	Unpinned dependencies	2	3	1	25	New
14	F-20	Any add treated as accumulation	2	3	1	25	New
15	F-12	Failures reported as REFUTED	3	3	2	24	T10
16	F-29	Vacuous and mock-only tests	3	3	2	24	New
17	F-31	Non-reproducible Docker build	2	4	2	24	New
18	F-28	Self-referential metrics	3	3	2	24	New
19	F-07	Initial value, max sentinel	3	4	3	21	T6
20	F-01	Proof ignores dataflow (core rewrite)	5	5	4	20	T5
21	F-14	Parametric pilot overclaims	2	3	2	20	New
22	F-19	Candidate ranking	2	3	2	20	New
23	F-23	Duplicated logic	3	2	2	20	New
24	F-30	CI breadth	2	2	1	20	New
25	F-16	min/max bounds	3	3	3	18	New
26	F-21	Parser silent information loss	3	3	3	18	New
27	F-26	Test data in runtime package	2	2	2	16	New
28	F-24	CLI module and option drift	3	2	3	15	New
29	F-25	Dead code	2	1	1	15	New
30	F-33	Repository hygiene	1	2	1	15	New
31	F-13	Unreachable exit code 2	1	2	1	15	New
32	F-22	Undeclared MLIR bindings	1	2	1	15	New
33	F-17	Replay crash	1	1	1	10	New

Rank	ID	Item	Impact	Risk	Effort	Priority	Tracked
34	F-34	Text-based MLIR, real semantics	3	3	5	6	New (research)

Remediation Plan

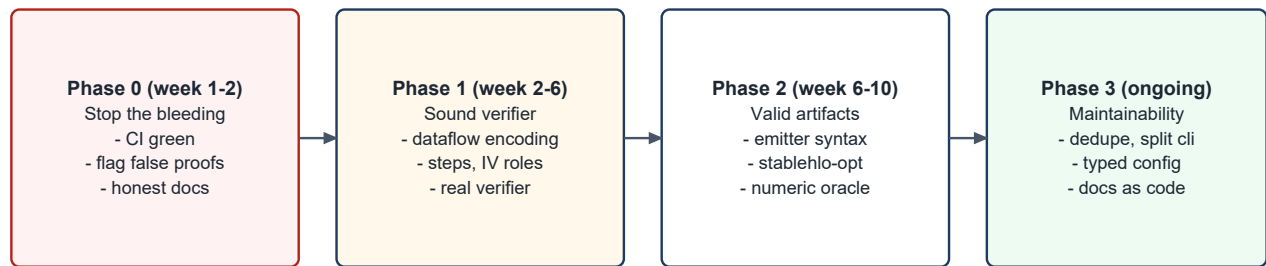


Figure: remediation phases. Phase 0 is a precondition for trusting any number the tool prints.

The plan is designed to run alongside the existing Plane cycles, with small PRs and strict-xfail tests landing first, as the team already agreed.

Phase 0: Stop the bleeding (weeks 1 to 2)

Goal: nothing the tool prints or the repository claims is misleading, and CI is green.

Step	Items	Plane	Owner lane	Done when
0.1	Land mutant regression tests as strict xfail, including the new canonical-fixture cases (elementwise_mul maps to mul, relu maps to relu) and the step fixture	T1, plus additions	C	CI collects all cases as xfail
0.2	Fix RecFunction naming and the conv recursive arity; stop discarding <code>ENCODE_ERROR</code> silently	T2	A	<code>test_lift_larger_matmul</code> passes; CI green
0.3	Quick guards: explicit payload branches for multiply/max/negate (F-02) and pre-filter rules for SCALE/NEGATE/MAX/MIN (F-18); reject steps other than 1 in <code>structural_match</code>	New N1, T7 step 2	A, B	canonical fixtures map to the right sketch or fail honestly
0.4	Trusted lane refuses the internal verifier; StableHLO marked UNVALIDATED unless <code>stablehlo-opt</code> exists	T9	C	new unmocked CLI test fails without a real verifier
0.5	Make <code>--no-verify</code> skip Z3; add ERROR and NO_MATCH states	T10	C	tests per T10
0.6	Put a "results under revision" banner at the top of <code>project-status.md</code> , plan v2 status sections, the proof-quality baseline and the paper findings file; link this handbook from <code>README.md</code> and <code>docs/README.md</code>	New N2	any	banner merged
0.7	Remove <code>.claude/settings.local.json</code> from git and ignore it	New N3	any	file untracked

Phase 1: A sound verifier (weeks 2 to 6)

Step	Items	Plane	Done when
1.1	Parser exact-name matching for constants and IVs	T3	mutant cases pass
1.2	Per-function extraction	T4	two-function mutant passes
1.3	Build proof terms from the SSA graph (three small PRs)	T5	arithmetic mutants pass, all good fixtures still prove
1.4	Model output initial values and max start values; refuse or condition accumulate-to-fresh matches	T6	new tests per T6
1.5	Loop steps in parser and proof	T7	step mutant passes
1.6	Numerical oracle over fixtures and mutants in CI	T11	oracle job green
1.7	Restate parametric pilot scope; give distinct extents distinct symbols; decide whether to keep it	New N4 (F-14)	notes and docs accurate
1.8	Token-based accumulation detection; parser failures with lost information mark the lift UNVERIFIED rather than PROVED	New N5 (F-20, F-21)	tests for each silent-loss diagnostic

Phase 2: Valid artifacts (weeks 6 to 10)

Step	Items	Plane	Done when
2.1	Fix <code>linalg.map</code> and <code>linalg.reduce</code> region syntax, <code>linalg.dot</code> rank-0 output, <code>conv_1d_ncw_fcw</code> rank; check <code>arith.maxf</code> against the MLIR version	New N6 (F-08)	every Linalg handler has an artifact-validation test with real <code>mlir-opt</code>
2.2	Refuse whole-tensor ops for partial loops; real scale constant	T8	T8 tests
2.3	Add <code>stablehlo-opt</code> (for example from the StableHLO release or IREE tools) to the Docker image and verifier selection per target	New N7 (F-09, extends T9)	StableHLO artifacts validated in CI
2.4	Parse affine <code>min/max</code> bounds or reject them explicitly	New N8 (F-16)	test with Polygeist-style bound
2.5	Rebuild the metrics pipeline on the batch command with profiles, validation and the new states; regenerate baselines; retire the letter grade until results are trustworthy	New N9 (F-28)	one committed baseline produced by CI

Phase 3: Maintainability (ongoing, alongside features)

Step	Items	Done when
3.1	Extract shared helpers: reduction iteration in <code>z3_checker.py</code> , transpose inference in <code>emitter.py</code> , Docker detection into one module, report helpers into <code>loophole/reporting.py</code> (F-23)	duplicate definitions removed
3.2	Split <code>cli.py</code> into a package (<code>cli/commands/*.py</code>) with shared option decorators and a typed <code>RunConfig</code> resolved once from flags, profile and environment (F-24)	option defaults consistent, documented in one table
3.3	Remove dead code and unused imports; single version source (F-25)	ruff clean
3.4	Move demo kernels out of <code>loophole.tests</code> (F-26)	<code>src/loophole/tests</code> gone
3.5	CI: Python 3.10 and 3.13 matrix, ruff, coverage report, <code>pip install -e .[dev]</code> , drop duplicate node-ID reruns, update actions (F-30)	CI matrix green
3.6	Pin dependencies with a lock or constraints file; single dependency declaration (F-32)	reproducible install
3.7	Dockerfile consumes <code>toolchain.lock</code> (checkout exact commits), pin base image digest, make every patch assert it applied, install LoopHole in the image (F-31)	rebuild from lock reproduces image
3.8	Replace conditional asserts; rename task-ID test classes by behaviour; reduce private-method tests (F-29)	review checklist
3.9	Mark historical docs clearly (front-matter banner), archive superseded plans, keep the handbook as the living reference (F-27)	docs index shows status per doc
3.10	Evaluate MLIR Python bindings for parsing and emission in a research experiment before committing to it (F-34)	ADR written

Proposed new Plane tasks (need confirmation before creation)

Proposed	Title	Findings	Lane	Size
N1	Z3: explicit payload branches and pre-filter rules so canonical fixtures stop mis-proving	F-02, F-18	A	small
N2	Docs: "results under revision" banners and handbook links	F-27, F-28	C	small
N3	Repo hygiene: untrack local assistant settings and stale generated files	F-33	C	small
N4	Parametric pilot: distinct symbols per extent, accurate notes and docs	F-14	A	small
N5	Parser: dataflow-based accumulation; information-losing diagnostics block PROVED	F-20, F-21	B	medium
N6	Emitter: valid syntax for map/reduce/dot/conv_1d with real-mlir-opt tests for every handler	F-08	B	medium
N7	Toolchain: add stablehlo-opt and per-target verifier selection	F-09	C	medium
N8	Parser: affine min/max bounds	F-16	B	medium
N9	Metrics: rebuild weekly/proof-quality reports on batch with new states	F-28, F-12	C	medium
N10	CLI: fix replay crash and exit-code contract without mocks	F-13, F-17	C	small

Working Agreements to Keep the Plot

The September process (small PRs, cross-lane review, strict xfail first, file ownership) is the right foundation. These additions target the specific failure modes seen in this codebase.

1. **A "proved" claim needs two independent witnesses.** Z3 and the numerical oracle must agree before a fixture's expected state is PROVED. If they disagree, the test fails.
2. **Every emitter handler has a real-toolchain test.** Adding a sketch requires: a correct fixture, at least one mutant that must not prove, and an artifact-validation test with the real verifier for its dialect.
3. **No silent fallback.** Any code path that drops information (truncating operands, defaulting types, discarding a loop clause, swallowing an exception) must either raise or downgrade the result state. Reviewers reject `except Exception: return <default>` without a downgrade.
4. **Numbers live in generated artifacts, not prose.** Docs link to a CI-produced report instead of copying counts. Remove test counts from lane records going forward.
5. **Name things by behaviour.** No task IDs in class names, function names, CI step names or code comments; put the task ID in the PR title and commit message only.
6. **Docs change with code.** If a PR makes a handbook or guide statement false, it updates the statement. The PR template gets a checkbox.
7. **Pin what affects results.** Z3, SymPy and the Docker toolchain versions are part of every published result's metadata and are changed only by a dedicated PR.
8. **AI-assisted changes get the same bar.** Generated code is welcome, but the author must be able to explain each changed line in review, and the PR must include a test that would fail without the change.

Appendix: Reproducing the Findings

All experiments were run from `packages/loophole` with `PYTHONPATH=src` (or the installed package) and `PYTHONUTF8=1`. A condensed version of the key reproductions:

```
from loophole.lifter import Lifter
from loophole.affine_extractor import AffineExtractor
from loophole.tests import fixtures as F
from loophole import mlir_validator as mv

HDR = ("func.func @k(%A: memref<4x4xf32>, %B: memref<4x4xf32>, %C: memref<4x4xf32>) {\n"
      "  affine.for %i = 0 to 4 {\n    affine.for %j = 0 to 4 {\n{body}    }\n  }\n  return\n}\n")
b_minus_a = HDR.replace("{body}",
      "    %a = affine.load %A[%i, %j] : memref<4x4xf32>\n"
      "    %b = affine.load %B[%i, %j] : memref<4x4xf32>\n"
      "    %s = arith.subf %b, %a : f32\n"
      "    affine.store %s, %C[%i, %j] : memref<4x4xf32>\n")
print(Lifter(target="linalg", strict_mode=True).lift(b_minus_a).summary())      # F-01: [OK] ... PROVED

print(Lifter(target="linalg").lift(F.ELEMENTWISE_MUL_MLIR).sketch_name)          # F-02: linalg.map{arit
      h.mulf_scalar}
print(Lifter(target="linalg").lift(F.RELU_MLIR).sketch_name)                    # F-02: linalg.copy
print(AffineExtractor().extract(F.MIXED_REALWORLD_MATMUL_MLIR).bounds)          # F-03: step 2 lost

r = Lifter(target="linalg").lift(F.ELEMENTWISE_ADD_MLIR)
print(mv.validate_mlir_artifact(r.emitted_mlir, mv.DOCKER_VERIFIER_CMD).stderr) # F-08: expected SSA op
      erand

import os; os.environ["LOOPHOLE_DISABLE_DOCKER_MLIR_VERIFY"] = "1"
print(mv.find_mlir_verifier())                                                  # F-10: loophole-intern
      al-mlir-verify
```